

CS 64 HANDOUT #4

MIPS Assembly of Instructions

Prof. Ziad Matni

*Read this handout **before** coming to class. In lecture, I will expand a lot on the concepts in this document, and give lots of examples. So, these handouts are a good explanation of what I'll be talking about in class, but they are not good substitutes for missing the lecture! ☺*

In this reading, we will discuss the following topics:

- R-Type Instructions
- I-Type Instructions
- Reverse assembly

Recall that each MIPS instruction is made up of 32 bits. *How* these bits are *organized* and put together (i.e., **assembled**) is the topic of this reading.

Let's start by examining the 3 basic instruction *formats*. Refer to your MIPS Reference Card for these 2 tables found on the front page:

R	opcode	rs	rt	rd	shamt	funct
	31	26 25	21 20	16 15	11 10	6 5 0
I	opcode	rs	rt	immediate		
	31	26 25	21 20	16 15	0	
J	opcode	address				
	31	26 25	0			

Each of the 3 types of instruction formats is shown here along with the organization of the 32 bits therein. We'll look at the R-types and I-types individually (we'll skip the J-types for now).

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	No

Note that this table tells you what *unique* numbers each of the 32 registers is given.

1. R-Type Instructions

Examples:

```
add $t0, $t1, $t2    # t1 + t2 → t0
sub $a0, $a1, $t6    # a1 - t6 → a0
sll $s1, $t0, 5      # t0 << 5 → s1
```

The R-Type format takes the 32 instruction bits and divides them up into **6 fields in this order**:

Field Name	Bit size	Bit No.s	Description
op code	6 bits	31 – 26	Unique code for basic operation (primary)
rs code	5 bits	25 – 21	Unique code for 1 st register source operand
rt code	5 bits	20 – 16	Unique code for 2 nd register source operand
rd code	5 bits	15 – 11	Unique code for destination register operand
shamt code	5 bits	10 – 6	Shift amount
funct code	6 bits	5 – 0	Unique code for basic operation (secondary)

The general syntax for most R-types, if you recall, is:

<operation> <r_destination> <r_source> <r_target>

So, looking at a familiar instruction like: `add $t0, $s1, $s2`

We note that the following:

1. The operation is **add**. The MIPS Reference Card table entitled “Core Instruction Set” tells us that **add** has **OP/FUNCT** values equal to **0/0x20**. This means that the **op code** field for this instruction, which is 6 bits in size, takes on the value **0** (or more precisely, **000000**), and the funct code field, which is also 6 bits, takes on the value **0x20** (or more precisely **100000**). IMPORTANT: note that **op code** is the 1st field in the list of ordered fields for R-types, and that **funct code** is the 6th (last) field.
2. The destination register is **\$t0**. The MIPS Reference Card table entitled “Register Name, Number, Use, ...” tells us that that **rd code** equals that register’s number, **8**. Since **rd code** is 5 bits in size, it is precisely, **01000**. IMPORTANT: note that **rd code** is actually the 4th field in the list of ordered fields for R-types.
3. The source register is **\$s1**. We gather that **rs code** equals that register’s number, **17**. Since **rs code** is 5 bits in size, it is precisely, **10001**. IMPORTANT: note that **rs code** is actually the 2nd field in the list of ordered fields for R-types.
4. The target register is **\$s2**. We gather that **rt code** = **18**. It too is 5 bits in size, therefore it’s precisely, **10010**. IMPORTANT: note that **rt code** is actually the 3rd field in the list of ordered fields for R-types.

- The remaining field is **shamt**, or **shift amount**. Since this instruction does not have a shift amount (this only applies to **sll**, **srl**, or **sra** instructions), its value is **00000**. This field is in 5th place.

When we put all this information together, then the ordering of fields for R-types dictates that the instruction **add \$t0, \$s1, \$s2** is assembled in 32-bits as:

```
000000 10001 10010 01000 00000 100000
op      rs      rt      rd      shamt funct
6b      5b      5b      5b      5b      6b      → Total is 32b
```

We usually express these 32-bit instructions as an 8-digit hexadecimal, so let's convert binary-to-hex:

```
Original: 000000 10001 10010 01000 00000 100000
Collect-4: 0000 0010 0011 0010 0100 0000 0010 0000
Hex value: 0x02324020
```

Let's look at another instruction: **sll \$t8, \$a0, 5**

We note that the following:

- The operation is **sll**. This has **OP/FUNCT** values equal to **0/0**.
- The destination register is **\$t8**. So, **rd code = 24**.
- The target register (**rt**) is **\$a0**. So **rt code = 4**.
NOTE: How do you know **\$a0** is **rt** and not **rs**?! The answer is to look carefully at the MIPS Reference Card entry for **sll** – you'll see the instruction described as:
R[rd] = R[rt] << shamt
- There is NO source register (**rs**) (instead **sll** has a **shamt** value). So **rs code = 0**.
- The shift amount is 5, so **shamt = 5**.

Assembling the bits, we get:

```
000000 00000 00100 11000 00101 000000
op      rs      rt      rd      shamt funct
6b      5b      5b      5b      5b      6b
```

Converting this to hex:

```
Original: 000000 00000 00100 11000 00101 000000
Collect-4: 0000 0000 0000 0100 1100 0001 0100 0000
Hex value: 0x0004C140
```

2. I-Type Instructions

Examples:

```
addi $t0, $t1, 13      # t1 + 13 → t0
andi $s1, $a1, 15      # a1 & 15 → a0
lw $a0, -4($t1)        # a0 = *( mem-location ( $t1 - 4 ) )
```

The R-Type format takes the 32 instruction bits and divides them up into **4 fields in this order**:

Field Name	Bit size	Bit No.s	Description
op code	6 bits	31 – 26	Unique code for basic operation
rs code	5 bits	25 – 21	Unique code for 1 st register source operand
rt code	5 bits	20 – 16	Unique code for register destination operand
immed code	16 bits	15 – 0	Immediate 16b (signed) value

The general syntax for most I-types, if you recall, is:

<operation> <r_target> <r_source> <immediate>

So, looking at a familiar instruction like: `addi $t0, $t1, 13`

We note that the following:

1. The operation is **addi**. The MIPS Reference Card table entitled “Core Instruction Set” tells us that **addi** has **OP** values equal to **0x8**. This means that the **op code** field for this instruction, which is 6 bits in size, takes on the value 8 (or more precisely, **001000**). There is no **funct code** for I-types...
2. The destination register is **\$t0**. So, **rt code** (it’s not called **rd code** in I-types) equals that register’s number, **8**. Since **rt code** is 5 bits in size, it is precisely, **01000**.
3. The source register is **\$t1**. We gather that **rs code** equals that register’s number, **9**. Since **rs code** is 5 bits in size, it is precisely, **01001**. IMPORTANT: Note that, in the prescribed ordering of the assembled bits, **rs code** field comes before the **rt code** field!!
4. The remaining field is **immediate**. The value for this in the instruction is **13**. Remember that this has to be expressed in 16 bits, so it is precisely: **0000 0000 0000 1101**.

When we put all this information together, then the ordering of fields for I-types dictates that the instruction `addi $t0, $t1, 13` is assembled in 32-bits as:

```
001000 01001 01000 0000 0000 0000 1101
op      rs      rt      immediate
6b      5b      5b      16b          → Total is 32b
```

In hex:

Collect-4: **0010 0001 0010 1000 0000 0000 0000 1101**

Hex value: **0x2128000D**

Let's look at another instruction: `lw $a0, -4($t1)`

We note that the following:

1. The operation is `lw`. This has **op code** value of `0x23`, or `10 0011` in 6-bit binary.
2. The destination register is `$a0`. So, **rd code** = `4`, or `0 0100` in 5-bit binary.
3. The source register is `$t1`. So **rs code** = `9`, or `0 1001` in 5-bit binary.
4. The immediate amount is `-4` (i.e., it's the **lw offset value**). This is a negative 16-bit number, so we should figure it out via 2's complement method.
So, since `4 = 0000...0100`, then `-4 = 1111...1100`

Assembling the bits, we get:

```
100011 01001 00100 1111 1111 1111 1100
op      rs      rt      immediate
6b      5b      5b      16b          → Total is 32b
```

In hex:

Collect-4: `1000 1101 0010 0100 1111 1111 1111 1100`

Hex value: `0x8D24FFFC`

3. Reverse Assembly (Disassembly) of Instructions

The knowledge of how to assemble MIPS instructions to 32 bits also allows you to do the reverse, or disassembly.

Say for example, you were told that a MIPS instruction's machine language value was

`0x01AE4821`. This means that, in binary, this is:

`0000 0001 1010 1110 0100 1000 0010 0001`

Where to begin?! The key here is that **ALL** MIPS instruction's **op codes** constitute the 1st 6 bits! In our example, the op code is: `000000`. A quick look at the MIPS Reference Card, illustrates that **ALL** R-type instructions have op codes equal to 0! What distinguishes one R-type instruction from another is the "secondary ID field" – the **funct code**. Since the **funct code** is always the last 6 bits, we can see from our example, that that is: `100001`, or `0x21` (it's useful to have in hexadecimal form because it makes it easier to look up). Finally, a reverse search of the table reveals that **OP/FUNC** entry for `0/0x21` is `addu`.

This is great information, because we know that `addu`, an R-type, has the syntax:

`addu rd, rs, rt`

We now need to identify these 3 registers.

Going back to the original, given binary value, we'll organize it according to the R-type field orders:

```
Original: 0000 0001 1010 1110 0100 1000 0010 0001
Organized: 000000 01101 01110 01001 00000 100001
           op      rs      rt      rd      shamt funct
           6b      5b      5b      5b      5b      6b
```

This quickly reveals that:

- **rs code** = 01101 = **13**, meaning that **rs = \$t5** (by reverse look-up of the registers table)
- **rt code** = 01110 = **14**, meaning that **rt = \$t6**
- **rd code** = 01001 = **9**, meaning that **rs = \$t1**
- **shamt code** = 00000, which is to be expected...

So, the disassembled instruction of **0x01AE4821** is **addu \$t1, \$t5, \$t6**.

5. Other Instructions

In this reading, we did *not cover* how several other instructions in the MIPS set get disassembled. This glaringly includes branching and jump instructions. This is because, in disassembling these particular instructions, we have to also properly interpret the *labels* into *actual memory addresses*. Furthermore, branching (**beq**, **bne**) does this differently from jumping (**j**) instructions and it involves calculating *relative addresses* versus *absolute addresses*. In our CS program, we have decided that this will be covered in the next course in this series on Computer Architecture. As such, you will not be responsible to know about how to do this for this course.